

CSC270 · DATABASE SYSTEMS · LECTURE 25

Physical Design — Query Performance

Indexing

CSC270 · Database Systems — COMSATS University Islamabad, Lahore Campus

Agenda

1

Why no index forces a full table scan

2

Primary indexes — built on the file's ordering key

3

Secondary indexes — for fields the file isn't sorted by

4

Dense vs. sparse indexes

5

Single-level vs. multi-level indexes (B-trees)

6

Real CREATE INDEX SQL syntax

7

Choosing which columns are worth indexing

8

The trade-off: faster reads, slower writes

The Cost of Not Having an Index

PropertyForRent with 10,000,000 rows across 500,000 disk blocks, and a query for propertyNo = 'PG21'.

WITHOUT AN INDEX — FULL TABLE SCAN

Read block 1

Check every row — no match

Read block 2

Check every row — no match

... up to block 500,000

Worst case: read every block

WITH AN INDEX — DIRECT LOOKUP

Look up PG21 in the index

A handful of comparisons, not 500,000

Follow the pointer

Jump straight to block 214,009

DEFINITION

An index is a small, separate structure mapping field values directly to the block/row that holds them — the DBMS jumps to the answer instead of scanning for it.

Primary Indexes

Built on the field the data file is physically ordered by — almost always the primary key. One entry per distinct key value, pointing to the first record with that value.

Primary index on PropertyForRent(propertyNo) — file ordered by propertyNo

Index entry: propertyNo	Pointer
PA14	→ block 1
PG4	→ block 2
PG21	→ block 2
PG36	→ block 3
PL94	→ block 3

ASSUMES AN ORDERED FILE

This only works because the file is physically stored in propertyNo order. If unordered (the common case), no primary index is possible — you'd need a clustering index or a secondary index instead.

Secondary Indexes

Most useful queries filter on a field the file isn't ordered by — e.g. “find all properties in Lahore.” A secondary index must have one entry for every record, since matching rows can be scattered anywhere in the file.

Secondary index on PropertyForRent(city)

Index entry: city	Pointer
Islamabad	→ PA14
Karachi	→ PL94
Lahore	→ PG4
Lahore	→ PG36
Lahore	→ PG21

WHY LAHORE REPEATS

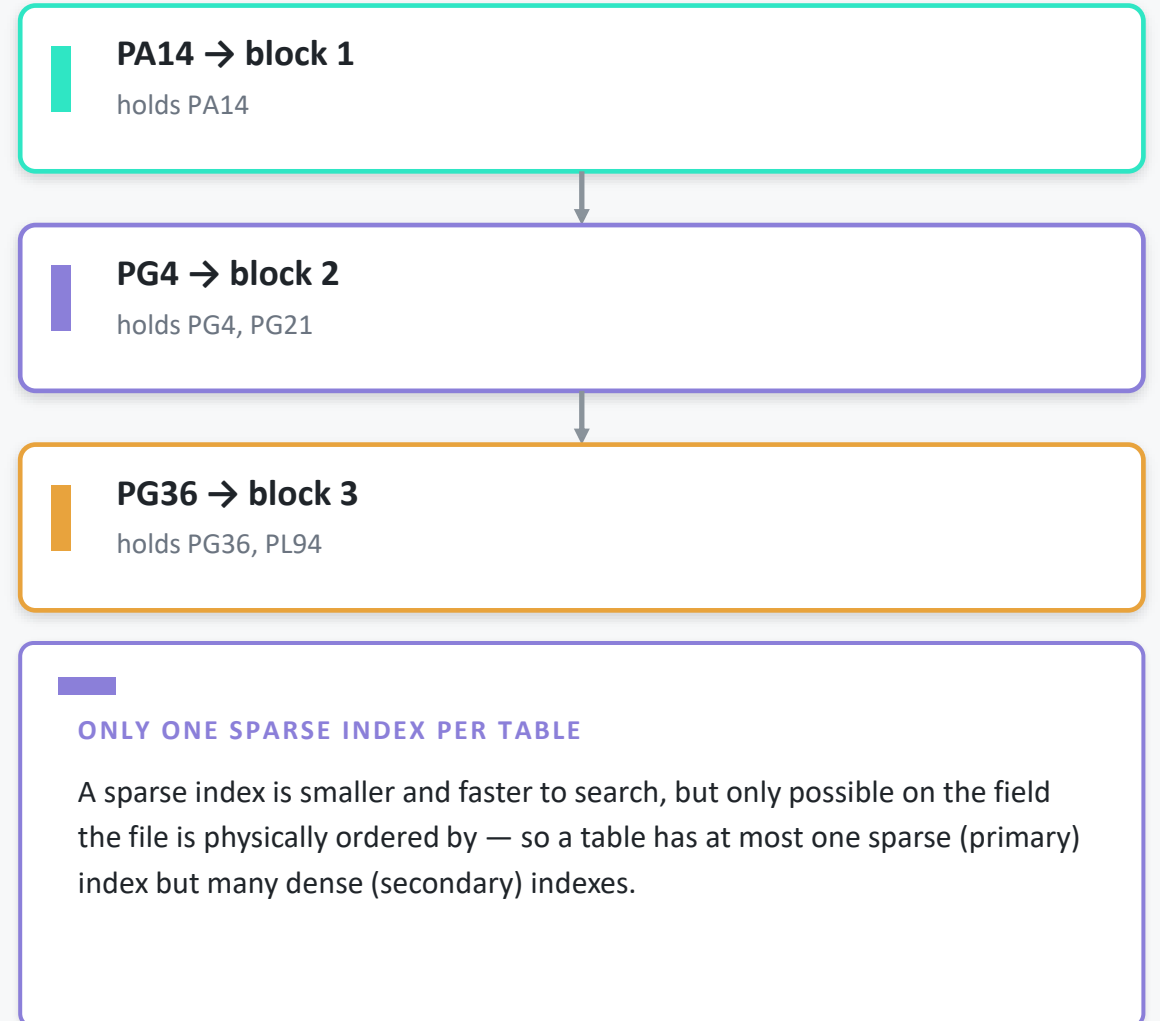
Lahore appears three times — once per matching record — because a secondary index can't compress multiple rows into one entry the way a primary index can. This is the direct cause of the dense-vs-sparse distinction next.

Dense and Sparse Indexes

DENSE INDEX — 1 ENTRY / RECORD



SPARSE INDEX — 1 ENTRY / BLOCK



Single-Level and Multi-Level Indexes

An index over ten million rows is itself a large file. The fix: build an index on the index — a multi-level index, the structure underlying every production B-tree/B+-tree.

Top-level (second-level) index

Small enough to fit in one block — one entry per block of the level below

First-level index

One entry per data block, exactly like a single-level index

Data file

The actual PropertyForRent rows, in disk blocks

WHY THIS IS $O(\log N)$

Each level divides the search space by roughly the same factor — the entries that fit in one block. That's exactly what gives B-tree indexes their logarithmic lookup cost, versus a full scan's linear $O(n)$ cost, even as a table grows into the billions of rows.

Creating Indexes in SQL

You never build index structures yourself — the DBMS does it once you name the column(s).



SQL

```
-- A simple secondary index on one column
CREATE INDEX idx_property_city
ON PropertyForRent (city);

-- A unique index -- also enforces that no two rows may share a value
CREATE UNIQUE INDEX idx_property_no
ON PropertyForRent (propertyNo);

-- A composite (multi-column) index -- useful for queries filtering on both
CREATE INDEX idx_property_city_rent
ON PropertyForRent (city, rent);

-- Removing an index that is no longer earning its cost
DROP INDEX idx_property_city;
```

Index Selection and Performance

● High selectivity

Many distinct values (propertyNo, email). A 2-value column like isActive barely benefits.

● Foreign keys

Almost always worth indexing — constantly used in joins.

● Columns in WHERE clauses

Filters run often, against large tables, are the clearest case.

● Columns in ORDER BY / JOIN

An index on the sort/join column can skip a separate sort step entirely.

INDEXING EVERYTHING IS NOT A SHORTCUT

Every index must be updated on every INSERT/UPDATE/DELETE that touches the column — over-indexing slows writes dramatically without benefit. Index selection is a real design decision driven by the actual query workload, not “more is always better.”

Advantages and Disadvantages of Indexing

ADVANTAGES

Much faster reads

$O(\log n)$ lookups instead of $O(n)$ scans

Faster sorting and joins

An indexed sort order can avoid a separate sort step

Enforces uniqueness

UNIQUE indexes double as a constraint

DISADVANTAGES

Slower writes

Every INSERT/UPDATE/DELETE must also update every affected index

Extra storage

An index over a large table can itself be sizeable

Maintenance overhead

Indexes can fragment and need periodic rebuilding

Key Takeaways

- With no index, the DBMS runs a full table scan for any lookup — cost grows linearly with table size.
- A primary index (one entry per key) needs an ordered file; a secondary index must be dense (one entry per record).
- Sparse indexes (one entry per block) only work on an ordered file's primary index — every secondary index is dense.
- A single-level index that grows too large is solved with a multi-level index — the idea behind B-tree/B+-tree structures.
- Choose indexes from real query workload — selectivity, foreign keys, WHERE/JOIN/ORDER BY columns — never every column.
- Indexing trades faster reads and joins for slower writes and extra storage — never “free” performance.